

4000万美金的套利市场：手把手教你写Polymarket套利机器人

原创 QuantML QuantML 2025年12月29日 16:51 上海



作为基于 Polygon 区块链构建的全球最大去中心化预测平台，Polymarket 采用独特的混合架构（链下撮合、链上结算）以及 Gnosis 条件代币框架（Conditional Token Framework, CTF），创造了传统金融市场中罕见的确定性套利机会。在 Polymarket 上，一些开发者正悄悄每月赚取 10K、50K，甚至 200K+ 的套利收益。

我们将从市场微观结构出发，深入解构二元市场（Binary Markets）与负风险市场（Negative Risk Markets）的定价机制，推导无风险套利的数学不等式。详细探讨如何利用 Python 构建高频交易系统，涵盖从 Gamma API 市场发现、WebSocket 实时订单簿维护、到利用 Relayer 实现无 Gas 费交易执行的全流程。特别针对 Polymarket 独特的“合并（Merge）”与“赎回（Redeem）”机制，提供了完整的交互代码范例，解决了官方 CLOB 客户端功能的缺失问题。

预测市场微观结构与套利原理

在深入代码实现之前，必须从底层理解 Polymarket 的资产属性与定价逻辑。不同于传统的体育博彩或中心化交易所，Polymarket 的每一笔交易本质上都是对 ERC-1155 标准代币的所有权转移，这些代币代表了对未来事件结果的索取权（Claim）。

1.1 Gnosis 条件代币框架（CTF）的数学公理

Polymarket 的技术底座是 Gnosis 开发的条件代币框架（CTF）。CTF 允许将任何未来事件（Event）映射为一组互斥且穷尽的结果（Outcomes）。对于最基础的二元市场，例如“比特币在 2025 年底前是否会突破 10 万美元？”，结果集合为 $\{Yes, No\}$ 。

CTF 的核心设计公理是**概率守恒**。在一个有效的预测市场中，所有互斥结果的价格（即隐含概率）之和必须等于 1 单位的抵押品价值（通常为 1 USDC）。这意味着，如果你持有该市场所有可能结果的各 1 份代币（即 1 份 Yes 和 1 份 No），无论未来发生什么，你都拥有了对 1 USDC 的绝对索取权。

这一公理可以表示为恒等式：

$$\sum_{i=1}^n Price(Outcome_i) = 1.00$$

其中 n 是结果的总数。对于二元市场，即：

$$Price(Yes) + Price(No) = 1.00$$

1.2 套利的定义与类型

当市场由于流动性碎片化、做市商报价延迟或非理性交易行为导致上述恒等式被打破时，套利机会 (Arbitrage Opportunity) 便产生了。在 Polymarket 上，套利主要分为两类：**市场内再平衡套利 (Intramarket Rebalancing Arbitrage)** 和 **组合套利 (Combinatorial Arbitrage)**。

1.2.1 市场内再平衡套利 (荷兰书)

这是最直观的套利形式，通常被称为“荷兰书” (Dutch Book) 套利。当所有互斥结果的市场卖单 (Ask) 价格之和小于 1.00 时，套利者可以买入所有结果，并立即合成 (Merge) 为 USDC，锁定无风险利润。

数学模型：

设 $P_{ask}(Yes)$ 为买入 Yes 的成本， $P_{ask}(No)$ 为买入 No 的成本。

若满足：

$$P_{ask}(Yes) + P_{ask}(No) < 1.00 - C_{friction}$$

其中 $C_{friction}$ 代表交易摩擦成本 (包括潜在的 Taker 费和链上 Gas 费)，则存在套利空间。

实例分析：

假设在“美联储 12 月降息”市场中：

- 订单簿显示 Yes 的最优卖价 (Best Ask) 为 0.35 USDC。
- 订单簿显示 No 的最优卖价 (Best Ask) 为 0.63 USDC。
- 总成本 = $0.35 + 0.63 = 0.98$ USDC。
- 套利操作：花费 0.98 USDC 购买 1 Yes + 1 No。
- 结算操作：调用 CTF 合约将 1 Yes + 1 No 合并为 1.00 USDC。
- 净利润： $1.00 - 0.98 = 0.02$ USDC (收益率 2.04%)。

1.2.2 负风险 (Negative Risk) 市场的套利逻辑

Polymarket 引入了“负风险”结构来处理多结果互斥事件 (如“2024 美国大选共和党提名人”)。在这种市场中，存在多个候选人 (如 Trump, Haley, DeSantis)，但只有一个获胜者。这意味着所有候选人的“Yes”价格之和应为 1，而任意一个候选人的“No”价格应等于“其他所有候选人 Yes 价格之和”。

由于 Polymarket 的流动性通常集中在少数热门候选人身上，长尾候选人的定价往往效率低下，导致负风险市场成为套利的高发区。

套利场景 A: Yes 价格总和小于 1

$$\sum_{i=1}^m P_{ask}(Yes_i) < 1.00$$

操作：买入所有候选人的 Yes 代币。

套利场景 B：No 价格偏差

在负风险市场中，买入候选人 A 的 “No” 代币，本质上等同于买入 “除 A 以外所有候选人的 Yes” 的合成头寸。如果市场定价出现偏差，即：

$$P_{ask}(No_A) < \sum_{j \neq A} P_{bid}(Yes_j)$$

或者更常见的是：

$$P_{ask}(No_A) + P_{ask}(Yes_A) < 1.00$$

这里需要特别注意的是，Polymarket 的负风险市场在底层技术实现上是通过 NegRiskAdapter 合约将多个独立的二元市场（每个候选人是一个独立的 Yes/No 市场）串联起来的。当你在前端买入 “Trump: No” 时，你实际上是在与一个流动性池交互，该操作在底层可能涉及复杂的代币铸造或销毁。对于自动化套利，理解这一层抽象至关重要。

1.3 混合去中心化架构的影响

Polymarket 并非纯粹的链上 DEX（如 Uniswap），而是采用了混合架构：

- **链下订单簿（Off-chain CLOB）**：订单的提交、撮合在链下服务器完成，这保证了高频交易的可能，类似于币安或纳斯达克。
- **链上结算（On-chain Settlement）**：一旦订单撮合成功，交易数据会被提交到 Polygon 区块链上的 CTF Exchange 合约进行原子结算。

这种架构对套利策略有双重影响：

1. **速度优势**：无需等待区块确认即可确知成交，允许毫秒级策略。
2. **状态同步挑战**：你的程序维护的 “余额” 必须同时考虑链上已确认的余额和链下已成交但未上链的余额。如果同步失败，可能导致资金不足无法下单。

技术基础设施与环境搭建

要捕获稍纵即逝的套利机会，必须构建一套低延迟、高可靠的交易基础设施。我们将详细介绍基于 Python 的开发环境搭建、关键库的选择以及与 Polygon 网络的连接配置。

2.1 开发环境依赖

推荐使用 Python 3.9 或更高版本，以利用最新的异步特性。核心依赖库如下：

库名称	用途	关键功能
py-clob-client	官方 CLOB 客户端	订单签名、下单、撤单、获取 API 凭证
web3.py	以太坊/Polygon 交互	链上合约调用（Merge/Redeem）、余额查询
requests	HTTP 请求	查询 Gamma API（市场元数据）
websocket-client	WebSocket 连接	订阅实时订单簿推送，维持本地 Orderbook 镜像
python-dotenv	环境变量管理	安全存储私钥和 API Key

库名称	用途	关键功能
aiohttp / asyncio	异步并发	实现高并发扫描和下单

安装命令：

```
Bash
```

```
pip install py-clob-client web3 requests websocket-client python-dotenv aiohttp
```

2.2 账户体系：EOA 与 Proxy 钱包

这是 Polymarket 开发中最容易混淆但也最关键的部分。Polymarket 支持两种账户模式，这对套利成本有决定性影响。

2.2.1 外部拥有账户（EOA）

即直接通过 MetaMask 生成的私钥控制的账户。

- **优点：**简单，直接控制。
- **缺点：**每次链上操作（如 USDC 授权、合并代币）都需要消耗 MATIC 作为 Gas 费。
- **套利适用性：**低。因为高频套利涉及频繁的资金合并，Gas 费会吞噬微薄的利润。

2.2.2 代理钱包（Proxy Wallet / Gnosis Safe）

Polymarket 为每个用户部署的一个智能合约钱包（Gnosis Safe）。

- **优点：**支持 **Relayer（中继器）** 机制。用户只需对交易进行签名（元交易），由 Polymarket 的 Relayer 服务器代为上链并支付 Gas 费。
- **缺点：**交互逻辑复杂，需要特定的签名类型。
- **套利适用性：**极高。这是实现零 Gas 成本套利的基础。

强烈建议：所有套利机器人都应基于 Proxy 钱包构建。在 py-clob-client 中，通过指定 `signature_type=1`（对于 Magic Link）或 `signature_type=2`（对于 Gnosis Safe）来启用代理模式。

2.3 节点与 RPC 配置

虽然 CLOB 交互通过 HTTP/WS 进行，但代币的合并（Merge）和赎回（Redeem）是链上操作。Polymarket 官方的 Relayer 会处理大部分 Gas，但在某些情况下（或为了监控链上状态），你仍然需要直接读取 Polygon 链数据。

- **公共 RPC：**<https://polygon-rpc.com>（免费，但速率限制严格，不稳定）。
- **私有 RPC：**推荐使用 Alchemy, Infura 或 QuickNode。对于高频套利，稳定的 RPC 连接是必须的，以防止在关键时刻（如合并资金以进行下一笔交易时）超时。

2.4 Gamma API 与 CLOB API 的分工

Polymarket 的数据层分为两部分，套利系统需要同时使用：

1. Gamma API (gamma-api.polymarket.com) :

- **性质**：只读，提供富文本元数据。
- **用途**：市场发现。获取所有活跃市场的 QuestionID, ConditionID, TokenID, Slug 等信息。
- **频率**：低频（如每分钟扫描一次）。

2. CLOB API (clob.polymarket.com) :

- **性质**：读写，处理订单。
- **用途**：获取实时价格（Level 2 数据）、下单、撤单。
- **频率**：高频（毫秒级）。

自动化套利系统的架构设计

一个成熟的自动化套利系统应包含四个核心模块：市场扫描器（Scanner）、行情监控器（Monitor）、执行引擎（Executor）和 结算引擎（Settler）。

3.1 市场扫描器（Scanner）

该模块负责从 Gamma API 拉取所有潜在的可交易市场，并过滤出具备流动性的目标。

筛选逻辑：

- active = true：仅关注活跃市场。
- closed = false：排除已结束的市场。
- enable_order_book = true：确保市场在 CLOB 上交易（排除旧的 AMM 市场）。
- **流动性过滤**：通过 volume 或 liquidity 字段过滤掉死盘。对于套利者，没有流动性的市场即使有价差也无法成交。

代码实现范例（Python）：

```
Python

import requests
import time

def fetch_arbitrage_candidates():
    """
    从 Gamma API 获取活跃且开启订单簿的市场列表
    """
    url = "https://gamma-api.polymarket.com/markets"
    markets =
    limit = 100
    offset = 0
```

```

while True:
    params = {
        "limit": limit,
        "offset": offset,
        "active": "true",
        "closed": "false",
        "enable_order_book": "true",
        "order": "volume:desc"# 按交易量排序, 优先关注热点
    }
    try:
        response = requests.get(url, params=params)
        response.raise_for_status()
        data = response.json()

        if not data:
            break

        markets.extend(data)

        # 如果返回数量少于 limit, 说明已到末尾
        if len(data) < limit:
            break

        offset += limit
        time.sleep(0.1) # 礼貌性限流

    except Exception as e:
        print(f"Error fetching markets: {e}")
        break

return markets

# 解析市场数据, 提取 Token IDs
def parse_market_metadata(market):
    # 二元市场通常有两个 Token (Yes, No)
    # 负风险市场可能有多個
    tokens = market.get('clobTokenIds',)
    condition_id = market.get('conditionId')
    question = market.get('question')

    return {
        "question": question,
        "condition_id": condition_id,
        "tokens": json.loads(tokens) if isinstance(tokens, str) else tokens
    }

```

3.2 行情监控与价差计算 (Monitor)

这是系统的核心。由于 REST API 的速率限制 (通常 10 requests/sec) , 对于数百个市场进行轮询是不现实的。必须使用 WebSocket 订阅关键市场的 book 频道。

套利判定算法:

对于任意给定的市场 M , 其结果集合为 $O = \{o_1, o_2, \dots, o_n\}$ 。

从 CLOB 获取每个结果 o_i 的最优卖单价格 Ask_i 和对应的数量 $Size_i$ 。

1. 计算单位成本:

$$Cost_{unit} = \sum_{i=1}^n Ask_i$$

2. 判断套利机会:

若 $Cost_{unit} < 1.00 - Threshold$ (Threshold 设为 0.005 或更高以覆盖潜在滑点) , 则触发信号。

3. 计算最大可套利量 (Capacity) :

由于每个 Ask 档位数量不同, 最大成交量受限于最薄弱的一环:

$$Capacity = \min(Size_1, Size_2, \dots, Size_n)$$

4. 深度穿透 (Depth Penetration) :

如果第一档流动性不足以满足预设的最小利润额, 算法应向后遍历 Orderbook 的 Level 2、Level 3... 重新计算加权平均成本。

WebSocket 监听代码范例:

Python

```
import json
import threading
from websocket import WebSocketApp

class OrderBookMonitor:
    def __init__(self, market_tokens):
        self.ws_url = "wss://ws-subscriptions-clob.polymarket.com/ws/market"
        self.market_tokens = market_tokens # Map: token_id -> market_info
        self.orderbooks = {} # Local mirror

    def on_message(self, ws, message):
        data = json.loads(message)
        # 处理快照或增量更新
        # 更新 self.orderbooks
        # 立即调用 self.check_arbitrage(market_id)

    def check_arbitrage(self, market_id):
        # 提取 Yes 和 No 的 Best Ask
```

```

# 计算 Sum
# 如果 < 1.0, 调用 Execution Engine
pass

def start(self):
    self.ws = WebSocketApp(self.ws_url,
                           on_message=self.on_message)

    # 订阅逻辑...

    self.ws.run_forever()

```

3.3 执行引擎 (Executor)

执行引擎负责将套利信号转化为链上的实际资产。

并发下单的重要性：

你必须同时买入 Yes 和 No。如果先后下单，存在“单腿风险”（Legging Risk）。例如，你买入了 Yes，正要买 No 时，No 的价格突然飙升，此时你不仅没赚到钱，反而持有了单边的风险敞口。

在 Python 中，应使用 `asyncio.gather` 或多线程来实现并发 HTTP 请求。

API 认证与签名：

Polymarket 的 API 需要较为复杂的认证头（L1/L2 Headers）。py-clob-client 封装了这部分逻辑。

Python 下单代码范例：

Python

```

import os

from py_clob_client.client import ClobClient
from py_clob_client.clob_types import OrderArgs, OrderType
from py_clob_client.constants import BUY

# 从环境变量加载私钥
key = os.getenv("PRIVATE_KEY")
chain_id = 137 # Polygon Mainnet
host = "https://clob.polymarket.com"

# 初始化客户端 (使用 Proxy 模式)
client = ClobClient(host, key=key, chain_id=chain_id, signature_type=1)
# 获取或派生 API Credentials
client.set_api_creds(client.create_or_derive_api_creds())

def execute_arbitrage(token_yes, token_no, price_yes, price_no, size):
    """
    执行双边下单
    """

    print(f"Exec Arb: Size {size} @ Yes:{price_yes} / No:{price_no}")

    order_yes = OrderArgs(

```

```

        price=price_yes,
        size=size,
        side=BUY,
        token_id=token_yes
    )

    order_no = OrderArgs(
        price=price_no,
        size=size,
        side=BUY,
        token_id=token_no
    )

    # 实际应用中应使用 Async Client 并发发送
    # 这里演示同步调用
    try:
        resp_yes = client.create_and_post_order(order_yes)
        resp_no = client.create_and_post_order(order_no)
        print("Orders placed:", resp_yes, resp_no)
        return True
    except Exception as e:
        print(f"Execution failed: {e}")
        return False

```

3.4 结算引擎 (Settler) : Merge 与 Redeem

这是大多数公开教程缺失的部分。买入成功后，你持有了一篮子代币，资金被占用。为了复利，必须尽快将其合并回 USDC。

由于 py-clob-client 暂不支持 Merge，我们需要使用 web3.py 直接调用 CTF 合约。

合约信息：

- **CTF 合约地址 (Polygon):** 0x4D97DCd97eC945f40cF65F87097ACe5EA0476045
- **ABI:** 必须包含 mergePositions 函数的定义。

核心代码：使用 Web3.py 进行合并

Python

```

from web3 import Web3
from web3.middleware import geth_poa_middleware
import json

# 配置 Web3
w3 = Web3(Web3.HTTPProvider("https://polygon-rpc.com")) # 建议换成 Alchemy
w3.middleware_onion.inject(geth_poa_middleware, layer=0)

# 加载 ABI (简化版)

```

```

CTF_ABI = '[{"name": "partition", "type": "uint256"}, {"internalType": "uint256", "name": "am"}]'
CTF_ADDRESS = "0x4D97DCd97eC945f40cF65F87097ACe5EA0476045"
USDC_ADDRESS = "0x2791Bca1f2de4661ED88A30C99A7a9449Aa84174" # Polygon USDC (Bridged)

contract = w3.eth.contract(address=CTF_ADDRESS, abi=json.loads(CTF_ABI))

def merge_positions_on_chain(condition_id, amount_wei, private_key):
    """
    直接在链上调用 mergePositions
    注意: 这需要支付 Gas。如果是 Proxy 钱包, 需要通过 Relayer 发送元交易 (更复杂)
    """
    account = w3.eth.account.from_key(private_key)

    # 构建交易
    # partition 对应二进制 01 和 10, 即 Outcome A 和 Outcome B
    tx = contract.functions.mergePositions(
        USDC_ADDRESS,
        b'\x00' * 32, # parentCollectionId always 0
        condition_id,
        , # Binary market partition
        amount_wei
    ).build_transaction({
        'from': account.address,
        'nonce': w3.eth.get_transaction_count(account.address),
        'gas': 200000,
        'gasPrice': w3.eth.gas_price
    })

    signed_tx = w3.eth.account.sign_transaction(tx, private_key)
    tx_hash = w3.eth.send_raw_transaction(signed_tx.rawTransaction)
    print(f"Merge TX sent: {w3.to_hex(tx_hash)}")
    return w3.to_hex(tx_hash)

```

关于 Gasless Merge (高级):

为了利用 Polymarket 的 Gas 补贴, 你不能直接像上面那样发送交易。你需要构建一个符合 EIP-712 标准的 Safe Transaction, 签名后发送给 Polymarket 的 Relayer API (POST /relay)。这涉及到 gnosis-py 库和较为复杂的签名构建过程, 通常是专业做市商的壁垒所在。

成本分析与风险管理

在纸面上计算出的利润, 在实际操作中极易被忽视的成本和风险侵蚀。

4.1 交易成本结构

目前 Polymarket 官方声明不收取平台交易费（0 Fees），但这并不意味着交易是免费的。

成本类型	描述	影响程度
Taker Fee	尽管目前为 0，但有迹象表明针对美国合规市场将引入 0.01% 的 Taker 费。	潜在风险
Spread / Slippage	买一价与买二价之间的价差。大额套利必然面临滑点。	高 。这是主要成本。
Gas Fee (Matic)	链上合并（Merge）和赎回（Redeem）的费用。	中 。若使用 Relayer 可免除。
Opportunity Cost	资金占用成本。如果合并交易在链上卡住，资金利用率下降。	低 （Polygon 确认快）。

4.2 速率限制 (Rate Limiting)

Polymarket CLOB API 对请求频率有严格限制：

- **公共端点**：10 requests / 10 seconds。
- **Authenticated (下单)**：50 requests / 10 seconds。
- **Burst**：允许短时间突发，但若持续超限会被封禁 IP。

应对策略：

1. **令牌桶算法**：在代码中实现限流器，确保每秒请求数不超过阈值。
2. **Websocket 优先**：尽量通过 WS 获取数据，减少 HTTP GET 请求。
3. **多 IP 轮询**：对于极高频的扫描，可能需要代理池（Proxy Pool），但这增加了基础设施复杂度。

4.3 执行风险 (Execution Risk)

单边成交（Legging Risk）是自动化套利的最大噩梦。

场景：

1. 机器人发现套利机会：Yes @ 0.40, No @ 0.58 (Sum = 0.98)。
2. 发送购买 Yes 订单 -> **成交**。
3. 发送购买 No 订单 -> **失败**（因为别人先买走了，或者有人撤单了）。
4. 此时，你持有 Yes 的裸头寸。如果市场走势对你不利，你必须以更低的价格卖出 Yes 止损，导致亏损。

缓解方案：

- **FOK (Fill Or Kill)**：使用 FOK 订单类型，确保要么全成交，要么全不成交。但在 Polymarket 目前的 API 中，主要支持 Limit Orders (GTC/GTD)。模拟 FOK 需要极快的撤单逻辑。
- **IOC (Immediate Or Cancel)**：部分成交后立即取消剩余部分。
- **超额收益阈值**：只在价差足够大（如 > 3%）时才出手，以此作为安全垫，即使单边成交被迫止损，也能覆盖部分损失。

5.1 负风险市场的高级套利

在多结果市场中，partition 参数的设置更为复杂。

假设一个竞选有 A, B, C 三人。

- 如果 $\sum P(Yes) < 1$ ，你需要买入 A, B, C 的 Yes。合并时，partition 为 `` (对应二进制 001, 010, 100)。
- 如果 $P(No_A) + P(Yes_A) < 1$ ，你买入 No_A 和 Yes_A 。此时 No_A 本质上是 B | C 的合成。合并时，partition 为 `` (对应 001 和 110，假设 A 是第一位)。

理解这种**位掩码 (Bitmask)** 逻辑是编写通用合并脚本的关键。

5.2 竞争格局分析

Polymarket 上每年被提取的套利利润约为 4000 万美元，且头部效应明显。

- **顶级套利者**：使用 Rust 或 C++ 编写客户端，直接监听 Mempool 或使用私有高速节点，利用 Relayer 架构实现零 Gas。
- **普通套利者**：使用 Python 脚本，面临几百毫秒的延迟，只能吃到“残羹冷炙”（较小的价差或较慢的市场）。

未来趋势：

随着机构做市商（如 Wintermute）的介入，简单二元市场的价差将无限趋近于 0。套利者的机会将更多转移到：

1. **复杂事件关联**：如“特朗普当选”与“税收政策通过”之间的条件概率套利。
2. **跨平台套利**：如果 Kalshi 或 PredictIt 开放 API 并解决资金通道问题，跨交易所套利将是巨大的蓝海。

结语

在 Polymarket 上进行自动化套利是一场不仅关乎金融数学，更关乎软件工程与区块链底层技术的竞赛。虽然简单的“捡钱”时代可能正在过去，但通过精细化的系统设计——特别是利用 Proxy 钱包优化 Gas 成本，以及深入挖掘负风险市场的复杂逻辑——依然能在这个快速增长的预测市场中分得一杯羹。

对于开发者而言，建议从**小额资金、二元市场**入手，跑通“发现-下单-合并”的全流程，再逐步引入并发控制和负风险策略。切记，代码中的每一个报错都可能意味着真金白银的损失，严格的测试网（Polygon Amoy）演练是必不可少的环节。

QuantML 是链接全球顶尖量化人才的高端社群，我们聚焦于机器学习在量化投资中的最前沿应用。

核心价值：

- **顶级圈层：** 社区涵盖头部机构从业者、知名私募创始人、机构量化负责人，基金经理，券商金工分析师、GitHub千星作者及顶会学者构成。
- **每日高价值内容：** 持续分享前沿论文、论文研报复现、模型代码、核心Alpha因子以及QuantML-Qlib框架等。

加入我们，与最强大脑同行，洞见量化未来。

深度研究 · 目录

上一篇

超越Citadel：HRT证明“延迟套利”已死

下一篇

Jane Street：万亿交易机器的内部运作

阅读 5290
